



TRAINING PROGRAMME



Lesson 2

TDD Great Habits

"I'm not a great programmer. I am just a good programmer with great habits."— Kent Beck

TDD Habits

Unit Tests Foundation principles

- Tests should test only one single behaviour each
- Only *one logical assertion* per test
- Tests must be *mutually independent* (they must run in any order)
- Don't mix *state* and *collaboration* assertions
 - ◆ state assertion -> when asserting on return value or object public property
 - ◆ collaboration assertion -> when verifying object interactions on a mock



TDD Habits

Unit Tests Foundation - do and don't do

DO

- Keep *separated* tests and production code
- Organise Unit Tests to mirror the production code structure

DON'T

- Never refactor with failing tests (Never on RED!)
- Do not use production data or dependencies for your tests; only use what you can control

If your tests are difficult to write or maintain, consider changing the design



TDD Habits

- **Start writing a RED test**

- Create more specific tests driving more generalization (*Triangulation*)
- Give tests meaningful, goal-oriented names deriving from behaviour and reflecting business domain
- Start from the assertion and work backwards
- Make sure it is failing for the right reason with meaningful feedback
- Structure tests in Arrange, Act, Assert blocks (without comments plz)
 - ◆ Arrange (aka Given) -> context setup with all necessary preconditions and input
 - ◆ Act (aka When) -> on the method under test
 - ◆ Assert (aka Then) -> that the expectations are satisfied



TDD Habits

- **Then make it GREEN**
 - Write the simplest code to pass the test
 - Consider using Transformation Priority Premise
 - Write any code for going to the refactoring phase quicker
 - You can always improve your code later, if you are in ALL GREEN state



TDD Habits

- **When ALL GREEN -> REFACTOR!!!**
 - Refactor aggressively and constantly
 - Treat your tests as first class code
 - Use IDE to refactor safely and quickly
 - Use the rule of three for duplication cleanup
 - ◆ Introduce abstractions when you face the third repetition
 - ◆ Keep in mind that **duplication is cheaper than abstraction in terms of coupling**



TDD Habits - FIRST principles

→ Fast

- ◆ They should run very often, hence they must be fast: one second matter here

→ Isolated

- ◆ They should be no dependency between tests, hence they must run in any order

→ Repeatable

- ◆ They should always have the same result when run multiple times (no flaky tests)

→ Self validating

- ◆ Only two states: red or green. Absolutely no manual or human interpretation.

→ Timely

- ◆ Must be written at the right time (->BEFORE the code they suppose to test)



TDD Habits - naming Test classes and methods

Express what the class should be able to do.

```
public class MyClassShould {  
    @Test public void  
    do_something_interesting() {  
    }  
}
```

Start with a verb.

Reads as a full sentence



TDD Habits - naming Test classes and methods

```
public class BankAccountShould{  
  
    @Test public void  
    have_balance_of_zero_when_created() {  
        BankAccount bankAccount = new BankAccount();  
  
        assertThat(bankAccount.balance(), is(0));  
    }  
  
    @Test public void  
    have_the_balance_increased_after_a_deposit() {  
given    BankAccount bankAccount = new BankAccount();  
  
when    bankAccount.deposit(10);  
  
then    assertThat(bankAccount.balance(), is(10));  
    }  
  
}
```



TDD Habits - test smells

- Not testing anything
- Excessive setup
- Too many assertions
- Test too long
- Checking internals
- Checking more than strictly necessary
- Working only on dev machine
- Testing or containing irrelevant information
- Exception swallowing in test
- Test not belonging logically to the Fixture
- Obsolete test
- Hidden functionality buried in the setup
- Bloated construction impeding test readability
- Unclear failing reason
- Conditional test logic



TDD Habits



String calculator kata - part I

1- Create a string calculator with a method `int Add(string numbers)`

The method can take 0, 1 or 2 numbers comma separated and will return their sum. For an empty string it has to return 0. For example `""` will return 0, `"1"` will return 1 and `"1,2"` will return 3.

2- Allow the Add method to handle an undefined amount of numbers

3- Allow the Add method to handle also new lines as separator.

For example `"1\n2,3"` will return 6, but `"1,\n2"` is not a valid input

4- Support different delimiters. To setup the delimiter prefix the input with double forward slash `["//"]` followed by the delimiter and `"\n"`. Example: `"//;\n1\n2;3"` will return 6

5- Throw `"negatives not allowed"` exception with the list of negative numbers found when input contains any negative number



TDD Habits



String calculator kata - part II

6- Ignore numbers bigger than 1000. For example “2,1002” *will return 2.*

7- Support delimiters of any size. To setup the delimiter prefix the input with double forward slash followed by the delimiter wrapped in squared brackets and “\n”. For example: “//[****]\n1\n2****3” *will return 6.*

8- Allow multiple delimiters with this format: “//[delim1][delim2]\n”. For example: “//[*][%]\n1*2%3” *will return 6.*

9- Make sure the class supports multiple delimiters of any size, not just one character.



TDD Habits



Bowling game kata

Write a program to calculate the score of a game of Ten-Pin Bowling.

Input: a string representing a bowling game (* scoring rules on the next slide).

Output: the score as integer.

Conventions:

- "X" indicates a *strike*
- "/" indicates a *spare*
- "-" indicates a *miss*
- "|" indicates a *frame boundary*
- The characters after the "||" are bonus balls

Examples:

"X|X|X|X|X|X|X|X|X|X|XX" => Total score = **300** [10 frames x 30]

"9-|9-|9-|9-|9-|9-|9-|9-|9-|9-||" => Total score = **90** [10 frames x 9]

"5/|5/|5/|5/|5/|5/|5/|5/|5/|5/||5" => Total score = **150** [10 frames x 15]

"X|7/|9-|X|-8|8/|-6|X|X|X|81" => Total score = **167**



TDD Habits



* The scoring rules:

- Each game, or "line" of bowling, includes ten turns, or "frames" for the bowler.
- In each frame, the bowler gets up to two tries to knock down all ten pins.
- If the first ball in a frame knocks down all ten pins, this is called a "strike". The frame is over. The score for the frame is ten plus the total of the pins knocked down in the next two balls.
- If the second ball in a frame knocks down all ten pins, this is called a "spare". The frame is over. The score for the frame is ten plus the number of pins knocked down in the next ball.
- If, after both balls, there is still at least one of the ten pins standing the score for that frame is simply the total number of pins knocked down in those two balls.
- If you get a spare in the last (10th) frame you get one more bonus ball. If you get a strike in the last (10th) frame you get two more bonus balls.
- These bonus throws are taken as part of the same turn. If a bonus ball knocks down all the pins, the process does not repeat. The bonus balls are only used to calculate the score of the final frame.
- The game score is the total of all frame scores.

